

Technical Assessment

This assessment is designed to evaluate your proficiency in production ML engineering (MLOps), with emphasis on methodology, implementation quality, reproducibility, and business-oriented decision making.

Case Timeline

This case is expected to be fulfilled between four to five business days with a total dedication of around 25 hours of focused work. We can give an extension of up to three extra business days having given advance notice.

Case Context

R2 is preparing to operate a production-grade churn modeling workflow for SMBs in Latin America. Developing churn models from historical observations using anonymized preprocessed features is a key competitive advantage given the specialized transactional data R2 has access to, and serves as an additional lever to reduce the exposure of capital to unnecessary losses. In production, new users arriving in periodic batches should be scored by these models using the raw probabilities derived from the inferences during the batch processing, so that we estimate the likelihood of churning and avoid giving a loan to these users prone to churn. In consequence, monitoring is a key piece to keep models up-to-date and invariant as much as possible to changes in transactional distributions, reducing the losses and lowering the churn rates.

The goal of this assessment is not to build the most complex model, but to demonstrate how you design and implement a reliable local ML system end-to-end. You will work with a real anonymized dataset and simulate batch scoring and monitoring as if this were a production pipeline.

Data Provided

An anonymized data package with files in Parquet and json formats not exceeding 700 MB with the following content:

- `real_data/train/features_train.parquet`: training feature matrix with anonymized time-series features (C1 to C180), one row per user.
- `real_data/train/target_train.parquet`: training labels (LABEL) corresponding, row-by-row, to `features_train.parquet`. In this setting, a LABEL of 1 means the sample is a churn one, and 0 a non-churn one.
- `real_data/train/metadata.json`: schema metadata for training files (column names and data types).
- `real_data/batches/features/*.parquet`: production-like batch inputs to make inferences, containing the `merchant_id` and features C1 to C180.

- `real_data/batches/labels/*.parquet`: ground-truth labels per batch for monitoring/evaluation purposes, containing the `MERCHANT_ID` and the `LABEL`. Two important considerations
 - Normalizing field names is considered a part of the assessment, you can safely assume that every time there is a `merchant_id` equal to a `MERCHANT_ID`, both represent the same user.
 - Batch labels may have partial coverage versus feature rows (not all merchants having features will necessarily have labels included) Knowing how to handle this asymmetrical labeling is part of the assessment.
- `real_data/dataset_summary.csv`: file-level summary (rows, columns, label rates, and label coverage per batch).
- `real_data/data_dictionary.md`: data definitions, join conventions, and dataset usage notes.
- `real_data/README.md`: complementary explanations regarding the data contained in `real_data`.

The training data is already feature-engineered in time-series tabular form:

- Each row represents one user.
- Each of the 180 feature columns represents one point in the user transaction-count time series.
- Time granularity is 12 hours (2 observations per day).
- Feature order is chronological: `C1` is the oldest point and `C180` the most recent point (90 days after `C1` occurred).
- As described above, the training targets are provided separately in `target_train.parquet`.

Case Sections

The technical assessment is composed of three parts: model lifecycle implementation, batch operations & monitoring, and communication, as indicated in the sections below.

Tools Allowed:

- **Programming:** Python, preferably.
- **Infra tools:** we encourage the usage of local executions and free tools to avoid proprietary data to be uploaded to unknown services or limitations in the tools consumption.
- **Containerization:** Docker is allowed and encouraged for local supporting services such as the model registry, tracking servers, and monitoring stack building, among others.
- **Documentation:** office tools such as Microsoft Excel / Google Sheets and PowerPoint / Google Slides (Canva is also valid)
- **Visualization:** any visualization tool you feel comfortable with.

Submission Format:

Submit a Google Drive folder (we recommend the name pattern to be defined as `r2_assessment_<your_name>` , for example, `r2_assessment_CatalinaBernal`) with the following content:

- Parts 1 & 2:
 - Report document with architecture decision-making explaining assumptions and key results.
 - Code notebooks/scripts developed (if Python is used, please structure the directories in such a way that are easy to follow with best-practices, and include a README file with exact setup and execution instructions).
 - Dependency definitions (requirements.txt, pyproject-toml, or any other requirements file).
 - Dashboard / plots files, if any
- Part 3:
 - Presentation or report files
 - Dashboard / plots files

Part 1: Model Training, Experiment Tracking, and Registry

We have already extracted some feature-based data from our transactions to model churn. In this first part of the assessment, you will need to build a baseline binary classifier using the provided training data by following these key instructions:

1. Load and validate the training data (features_train + target_train).
2. Train at least one binary classifier for LABEL. The technique you use should reflect your modeling decision; even though we will not judge model performance nor model complexity first hand, we are expecting you to explain modeling decisions with clear arguments. Please consider that we value interpretability and explainability more than model complexity and performance when making business decisions.
3. Track experiments you execute, such as parameters, hyperparameters, performance metrics, and artifacts, among others.
4. Register the selected model version in a local model registry that would serve for later monitoring purposes. This will be the model to productionize.
5. Document the model selection logic in detail.

What we are looking for:

- Clear and reproducible training workflow.
- Correct registry integration (not only logging).
- Pragmatic model and performance metric selection.
- Prioritization of completeness, reproducibility, and production-oriented methodology over model complexity.

Part 2: Batch Inference and Monitoring

Now that you have a functional model trained, validated and tested, with all artifacts needed, in this second part you will implement a batch inference pipeline and a monitoring workflow. Please follow these instructions to guide your way through:

1. Develop a batch inference pipeline consuming data from real data/batches/features/.

2. Make inferences over the batched data using the model already registered.
3. Monitor model outputs, including the following (at least):
 - a. **Data quality** - You will define what quality means according to data distributions.
 - b. **Data/feature drift** - You will define what drift means according to the business case, and data distributions.
 - c. **Prediction and label drift** - You will define what drift means according to the business case, and prediction data distributions.
 - d. **Model performance** when the inference and ground-truth labels are available.
4. Create a prediction-output metadata file with the batch identifier, the run timestamp, the model version and the row count, among other metrics that we will let you include as complementary information.
5. Based on the metrics monitored, create a status coding framework (e.g. GREEN, AMBER, RED) to diagnose the model on a given time-frame that you should propose.
6. Take a couple of minutes to think about how you would design the system at scale, define retraining cycles, and orchestrate the system. This will have an impact on your implementation decisions.

What we are looking for:

- Operational robustness in batch execution.
- Traceability between model versions, batch inputs, predictions, and monitoring outputs.
- Clear thresholds and interpretable monitoring decisions.

Part 3: Data Storytelling & Communication

You can select one of the following two deliverables for this section:

- Prepare a short (8-slide hard limit) deck summarizing your developments from Parts 1 & 2.
- Prepare a non-technical report (6-page hard limit) describing your developments from Parts 1 & 2.

Imagine you're presenting this to the Risk and Product teams—keep the language business-friendly, action/impact oriented, and use visuals whenever you deem necessary (visuals should be done using a dashboarding tool or interactive visualization library, such as Tableau, Power BI, Looker Studio, or other).

We would like to see the following topics included in the deliverable:

- Problem statement
- Modeling strategy
- Architecture overview
- Key implementation decisions
- Monitoring strategy, drift methodology and results
- Known limitations and improvement opportunities
- Next steps

What we are looking for:

- Executive communication
- Clarity when communicating technical topics and ability to translate these into business impact
- Decision rationale and tradeoff awareness

Thanks again for your interest in joining R2. We are excited to see how you approach these challenges and uncover the insights that matter. We encourage you to take a thoughtful and creative solution—this is your chance to showcase how you think from an Engineering point of view, not just what you know. If you have any questions or need clarification at any point, please do not hesitate to reach out to Catalina Bernal, our hiring manager. We are rooting for you, good luck!